

BIRD MONITORING PROJECT

Data Manipulation, Cleaning & Interactive Dashboard Analysis

Complete Professional Project Report

This report combines the project data-preparation workflow with the final dashboard analysis into one continuous, presentation-ready document.

Part I - Data Manipulation & Cleaning

Part II - Dashboard Charts & Integrated Insights

Report flow

Source data -> Cleaning & validation -> Feature engineering -> Dashboard -> Insights

Prepared by

Fateh Mohammed Sayyed

Professional project documentation for review, viva, interview and presentation use

BIRD MONITORING PROJECT

Data Manipulation & Data Cleaning Code Report

Prepared by Fateh Mohammed Sayyed

Project stage	Purpose
Source preparation	Prepare a consistent analytical dataset from the supplied bird-monitoring records
Data manipulation	Standardize, validate, clean, de-duplicate and prepare fields for analysis
Time preparation	Create reusable year, month and weekday dimensions
Boundary	This report documents data-manipulation code only; chart/pie-chart code and chart data are intentionally excluded

Report scope

The objective is to explain the code used to transform the project dataset into a consistent, analysis-ready table. The report focuses on the manipulation logic: loading, inspection, missing-value assessment, standardization, empty-row/column removal, date conversion, text cleaning, blank-value normalization, duplicate-row handling, date-based feature engineering and final validation.

1. Project Data-Manipulation Workflow

The manipulation stage is deliberately performed before visualization. The supplied workflow creates a clean analytical base first, so later analysis works from consistent fields.

Step	Code operation	Why it is done
1	Load dataset	Bring the prepared CSV into pandas
2	Inspect schema and types	Understand columns and data types before changing them
3	Profile missing values	Quantify missingness and prioritize cleaning
4	Standardize columns	Make column references consistent and predictable
5	Remove empty rows/columns	Eliminate records/fields containing no information
6	Convert dates	Enable reliable temporal analysis
7	Clean text and blanks	Prevent category fragmentation caused by spaces/empty strings
8	Remove duplicates	Prevent repeated records from inflating counts
9	Create time features	Create reusable year/month/month-name/day fields
10	Validate output	Confirm the cleaned dataset is ready for analysis

Key principle

The code treats data cleaning as a controlled transformation: the original dataframe is copied, changes are applied to the copy, and quality is checked again after cleaning. This makes the manipulation process easier to audit and safer to reproduce.

2. Loading and Initial Inspection

The dataset is loaded with pandas using `read_csv(..., low_memory=False)`. The code also checks the shape and previews the first records. This gives the analyst an immediate view of dataset size and structure before manipulation begins.

Why this matters: The preview is diagnostic only: it is used to understand the incoming data, not to produce a final chart.

```
1 # Load the dataset produced by the project pipeline
2 file_path = "result_final(1).csv"
3
4 df = pd.read_csv(file_path, low_memory=False)
5
6 print(f"Dataset shape: {df.shape[0]:,} rows × {df.shape[1]:,} columns")
7 display(df.head())
```

3. Schema and Data-Type Inspection

The code prints the column names, creates a readable table of dtypes, and runs `df.info()`. This establishes which fields are text, numeric, or date-like and helps identify fields that need conversion before analysis.

Why this matters: This step is important because later operations such as datetime extraction depend on correct field types.

```
1 print("COLUMN NAMES:")
2 print(df.columns.tolist())
3
4 print("\nDATA TYPES:")
5 display(df.dtypes.to_frame("Data Type"))
6
7 print("\nDATASET INFORMATION:")
8 df.info()
```

4. Missing-Value Profiling

The missing-value block counts nulls for every column, sorts the results from highest to lowest, calculates a missing percentage, and displays only fields where missing values exist.

Why this matters: The purpose is to measure data quality before deciding what to clean. It avoids blindly filling or deleting values without first understanding the missingness pattern.

```
1 missing_before = (  
2     df.isnull()  
3     .sum()  
4     .sort_values(ascending=False)  
5     .to_frame("Missing Values")  
6 )  
7  
8 missing_before["Missing Percentage"] = (  
9     missing_before["Missing Values"] / len(df) * 100  
10 ).round(2)  
11  
12 display(missing_before[missing_before["Missing Values"] > 0])
```

Manipulation principle

The code changes the structure and quality of the dataset, not the meaning of the underlying observations. Cleaning is intended to make the records more consistent and comparable.

5. Core Data Cleaning

This block applies the main transformations: copy the dataframe, standardize column names, remove fully empty rows and columns, convert the date field with errors="coerce", trim text values, turn empty strings into NA, check duplicate rows, remove duplicates, and remove records with missing dates for temporal analysis.

Why this matters: This is the core manipulation stage. Each operation addresses a specific source of inconsistency that could otherwise distort grouping, filtering or time-based analysis.

```
1 # Work on a copy of the original dataset
2 clean_df = df.copy()
3
4 # -----
5 # 5.1 Standardize column names
6 # -----
7 clean_df.columns = (
8     clean_df.columns
9     .str.strip()
10    .str.lower()
11    .str.replace(" ", "_", regex=False)
12    .str.replace("-", "_", regex=False)
13 )
14
15 # -----
16 # 5.2 Remove completely empty rows and columns
17 # -----
18 clean_df = clean_df.dropna(axis=0, how="all")
19 clean_df = clean_df.dropna(axis=1, how="all")
20
21 # -----
22 # 5.3 Convert date column
23 # -----
24 if "date_x" in clean_df.columns:
25     clean_df["date_x"] = pd.to_datetime(
26         clean_df["date_x"],
27         errors="coerce"
28     )
29
30 # -----
31 # 5.4 Clean text columns
32 # -----
33 text_columns = clean_df.select_dtypes(
34     include=["object", "string"]
35 ).columns
36
37 for col in text_columns:
38     clean_df[col] = (
39         clean_df[col]
40         .astype("string")
41         .str.strip()
42     )
43
44 # -----
45 # 5.5 Replace empty strings with missing values
46 # -----
47 clean_df = clean_df.replace(r"^\s*$", pd.NA, regex=True)
48
49 # -----
50 # 5.6 Check and remove duplicate rows
51 # -----
52 duplicates_before = clean_df.duplicated().sum()
53
54 clean_df = clean_df.drop_duplicates()
55
56 duplicates_removed = duplicates_before
57
58 # -----
59 # 5.7 Remove rows with missing dates for temporal analysis
60 # -----
61 if "date_x" in clean_df.columns:
62     clean_df = clean_df.dropna(subset=["date_x"]).copy()
63
64 print("Cleaning completed successfully.")
65 print(f"Duplicate rows removed: {duplicates_removed}")
66 print(f"Clean dataset shape: {clean_df.shape[0]:,} rows x {clean_df.shape[1]:,} columns")
```

Manipulation principle

The code changes the structure and quality of the dataset, not the meaning of the underlying observations. Cleaning is intended to make the records more consistent and comparable.

6. Post-Cleaning Validation

After cleaning, the code recalculates missing values and missing percentages, checks that duplicate rows remaining are zero, and displays the first cleaned records.

Why this matters: The logic is important: cleaning is not considered complete until the output is checked again.

```
1 missing_after = (  
2     clean_df.isnull()  
3         .sum()  
4         .sort_values(ascending=False)  
5         .to_frame("Missing Values")  
6 )  
7  
8 missing_after["Missing Percentage"] = (  
9     missing_after["Missing Values"] / len(clean_df) * 100  
10 ).round(2)  
11  
12 print(f"Duplicate rows remaining: {clean_df.duplicated().sum():,}")  
13 display(missing_after[missing_after["Missing Values"] > 0])  
14  
15 display(clean_df.head())
```

7. Time Feature Engineering

The code derives year, month, month_name and day_of_week from the cleaned date field. These fields convert one raw date into reusable analytical dimensions.

Why this matters: This prepares the dataset for time-based grouping without repeatedly parsing dates during later analysis.

```
1 if "date_x" in clean_df.columns:
2     clean_df["year"] = clean_df["date_x"].dt.year
3     clean_df["month"] = clean_df["date_x"].dt.month
4     clean_df["month_name"] = clean_df["date_x"].dt.month_name()
5     clean_df["day_of_week"] = clean_df["date_x"].dt.day_name()
6
7 print("Time features created.")
8 display(clean_df[["date_x", "year", "month", "month_name", "day_of_week"]].head())
```

8. Final Manipulation Summary

The summary stage reports the final shape and key quality indicators after the manipulation pipeline. It provides a simple audit point before the data is passed to downstream analysis.

Why this matters: The final dataset is now structured for the dashboard and exploratory analysis; chart construction itself is outside the scope of this report.

```
1 summary = {
2     "Total Records": len(clean_df),
3     "Total Columns": clean_df.shape[1],
4     "Unique Species": clean_df["scientific_name"].nunique() if "scientific_name" in clean_df.columns else np.nan,
5     "Male Records": (clean_df["sex"] == "Male").sum() if "sex" in clean_df.columns else np.nan,
6     "Female Records": (clean_df["sex"] == "Female").sum() if "sex" in clean_df.columns else np.nan,
7     "Start Date": clean_df["date_x"].min().date() if "date_x" in clean_df.columns else np.nan,
8     "End Date": clean_df["date_x"].max().date() if "date_x" in clean_df.columns else np.nan
9 }
10
11 display(pd.DataFrame(summary.items(), columns=["Metric", "Value"]))
```

9. Professional Data-Preparation Conclusion

The project uses a reproducible manipulation pipeline in which the dataset is inspected first, then systematically cleaned and validated. The most important engineering choices are consistent column naming, controlled date conversion, text normalization, duplicate removal, explicit handling of empty records, and post-cleaning verification.

The result is a stable analytical foundation for the Bird Monitoring Dashboard. The dashboard can then perform filtering and aggregation on prepared fields without mixing core cleaning logic with presentation logic.

Important scope note

This report intentionally does not document the code used to build bar charts, donut charts, pie charts, or other visual outputs, nor does it reproduce their chart data. The focus here is strictly on data manipulation and preparation code.

Project owner

Fateh Mohammed Sayyed

BIRD MONITORING DASHBOARD

Detailed Chart-by-Chart Analysis and Integrated Insights

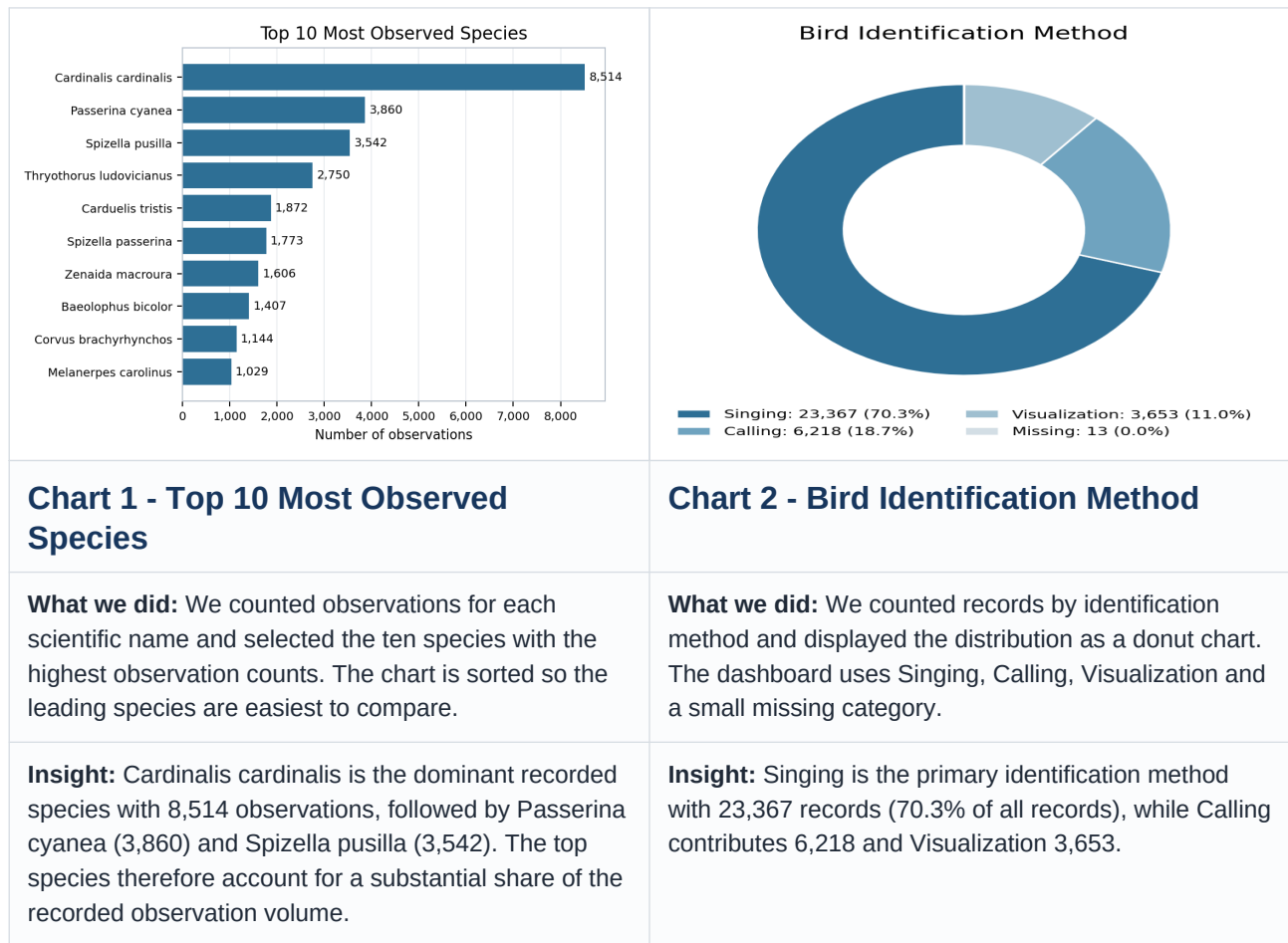
This report explains every visualization in the supplied dashboard, shows what was calculated, connects related charts, and translates the results into professional analytical insights. The calculations below reproduce the dashboard logic from the provided project code.

Total observations	33,251	Unique species	78
Recorded male	12,440	Recorded female	542
Identification methods	4	Sex categories	3

How to read this report

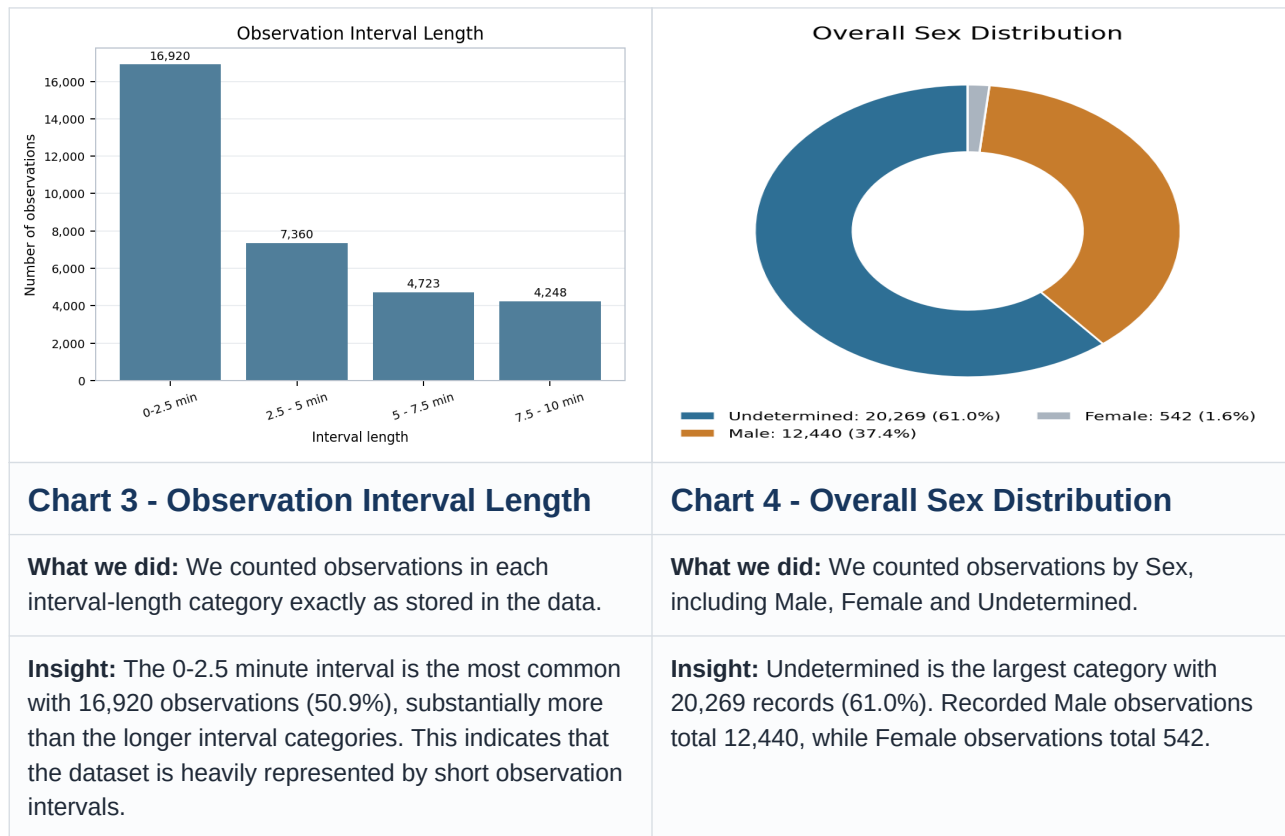
Each page follows the same structure: (1) the chart itself, (2) what we did, (3) what the chart tells us, (4) the analytical insight, and (5) how the paired chart strengthens the interpretation. This is designed for project reviews, viva, interview preparation, and stakeholder reporting.

1. Species Frequency + Identification Method



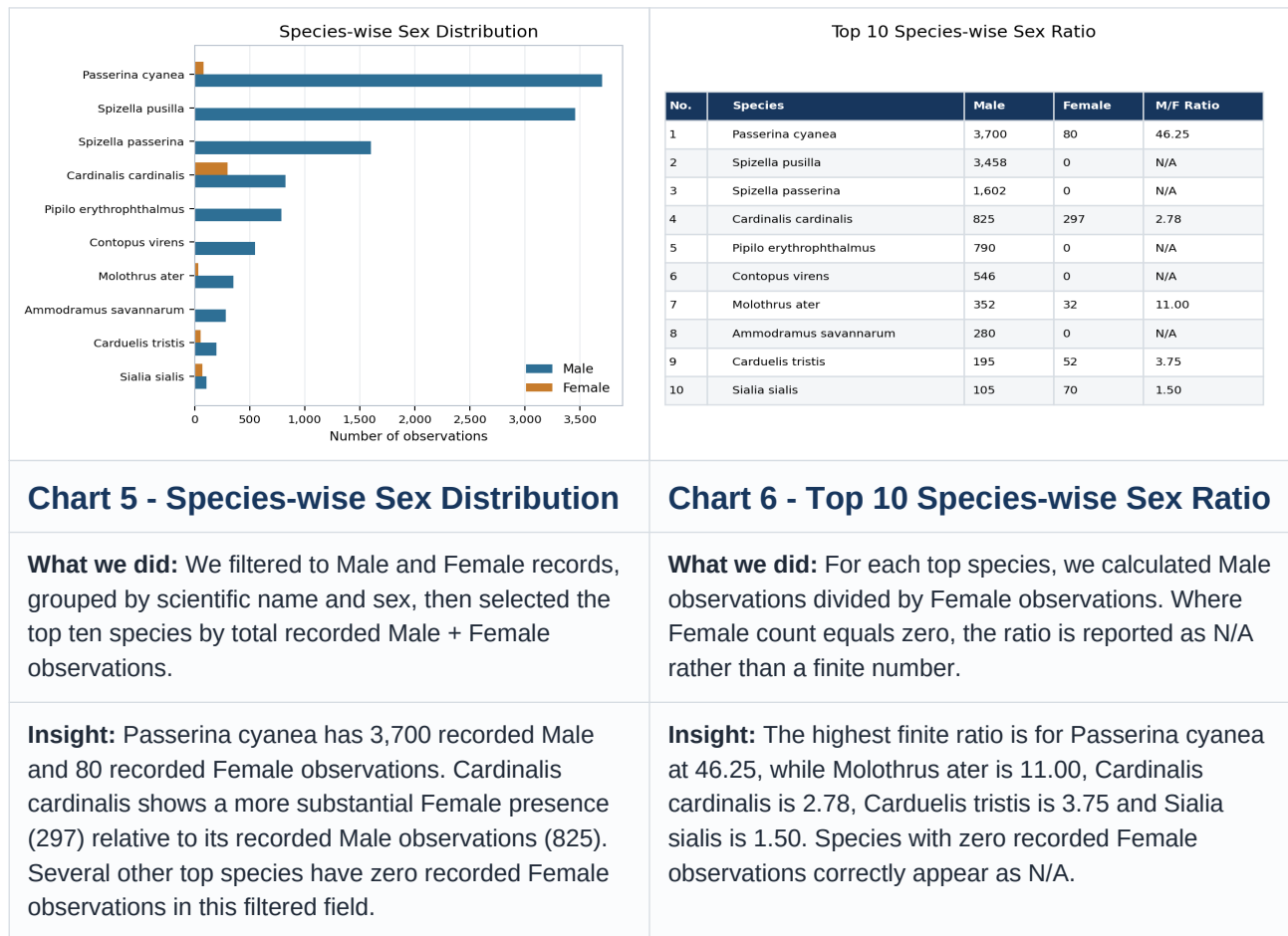
Integrated insight: Together, these charts connect **what is most frequently observed** with **how observations are identified**. This gives a stronger narrative than either chart alone: the dataset is concentrated around a small group of frequently recorded species, and the observation process is strongly represented by singing-based identification.

2. Observation Interval + Overall Sex Distribution



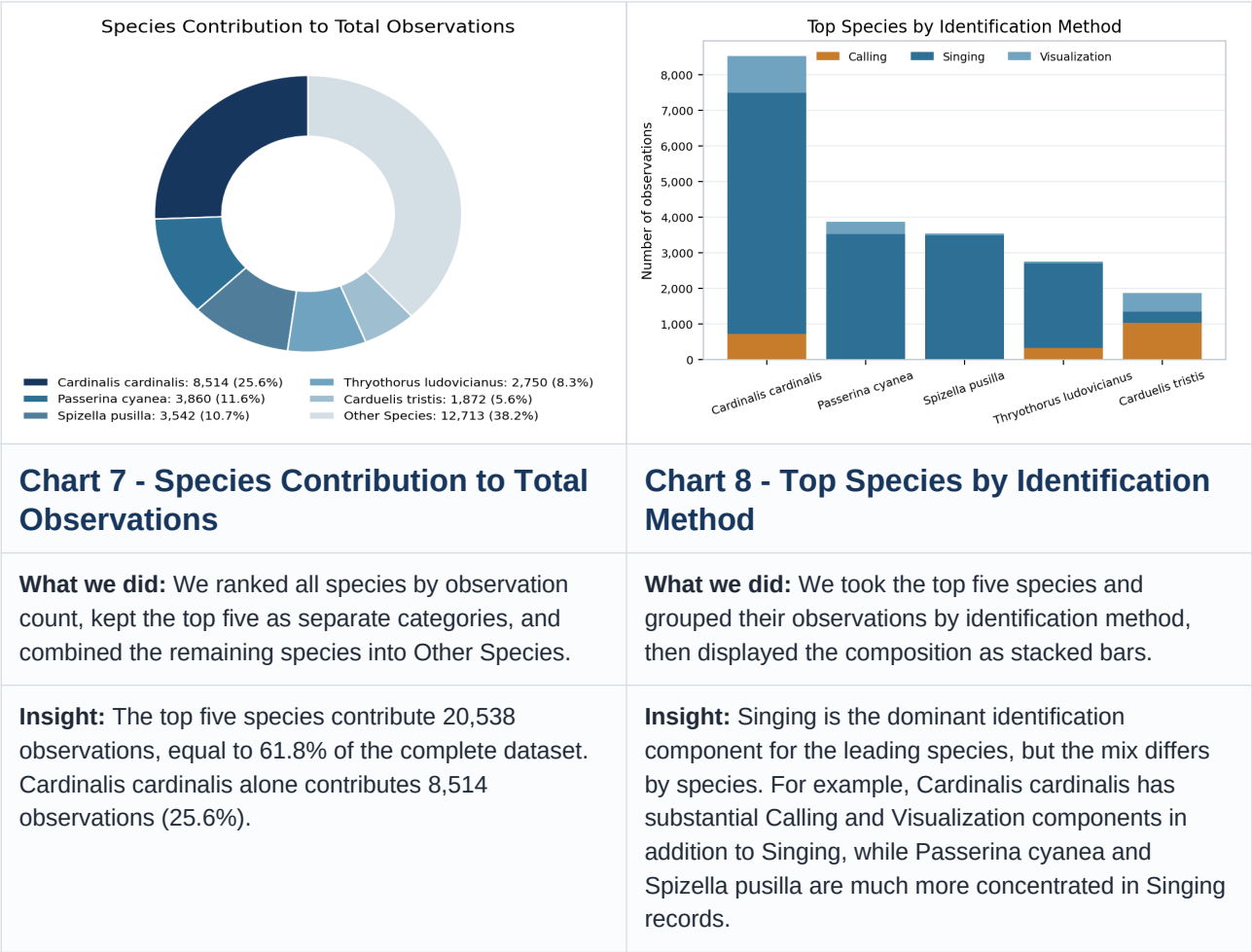
Integrated insight: These charts should be interpreted together because observation structure affects how we read downstream counts. The dataset is dominated by short intervals and also contains a large undetermined-sex category, so sex-based conclusions must be treated as **recorded-observation patterns**, not direct population estimates.

3. Species-wise Sex Distribution + Sex Ratio



Integrated insight: This is the dashboard's most detailed sex analysis. The grouped bars show the **absolute difference** between Male and Female observations, while the ratio table converts that difference into a standardized comparison. Together they move from visual magnitude to a numerical indicator. The ratio still reflects **recorded observations**, not biological population structure.

4. Species Contribution + Identification Method by Species



Integrated insight: These charts connect **species concentration** with **method composition**. The first chart tells us which species drive total observation volume; the second explains how the leading species are distributed across identification methods. This combination is especially useful for monitoring quality and understanding where observation effort is concentrated.

Professional Dashboard Conclusion

The dashboard is structured as a connected analytical story. It begins with observation volume and species dominance, adds the identification-method dimension, then introduces observation interval structure and overall sex composition. It becomes more granular through species-wise sex comparisons and finishes by linking species concentration with identification-method composition.

What we did: We transformed a large observation table into a small set of focused visual questions. Each chart has a specific calculation, and the paired charts are deliberately connected so that one visual adds context to the other.

What we learned: The supplied dataset contains 33,251 observations across 78 scientific species. The observation volume is concentrated in several species, Singing is the dominant identification method, short observation intervals are most common, and Undetermined is the largest sex category. At species level, recorded Male observations are often much higher than recorded Female observations for the top species, but this is an observation pattern rather than a population estimate.

Strengths: Interactive filtering, clear KPI summary, multi-dimensional analysis, species-level drill-down, and direct comparison of observation volume with identification and sex dimensions.

Limitations: The dashboard describes recorded observations. Differences in observation effort, interval duration, identification method, missing sex classification, and detection can influence the counts. Therefore, the charts should be used for monitoring and exploratory analysis rather than interpreted as direct estimates of ecological abundance or population sex ratios.

Interview-ready closing: "The key value of our dashboard is that we did not stop at creating charts. We connected the charts to answer a sequence of analytical questions: which species dominate the observations, how those observations are identified, how the monitoring intervals are structured, how sex is represented across species, and how the leading species contribute to the total dataset."

Source basis: supplied Streamlit dashboard code and supplied result_final(1).csv dataset. Calculations in this report reproduce the dashboard fields and aggregation logic shown in the project code.